Predicting Hospital Inpatient Costs Using Machine Learning: A Multi-Model Approach with Explainability and Production Deployment

Karthik Mulugu
karthikmulugu14@gmail.com

April 2026

Abstract

Accurate prediction of hospital inpatient costs is a critical challenge in healthcare economics, impacting resource allocation, insurance pricing, and patient financial planning. This paper presents a comprehensive machine learning system trained on the New York State SPARCS (Statewide Planning and Research Cooperative System) 2012 hospital inpatient discharge dataset, comprising approximately 2.5 million de-identified records. We evaluate five regression models—Ridge Regression, Random Forest, XGBoost, LightGBM, and a PyTorch Multilayer Perceptron (MLP)—with systematic hyperparameter optimisation via Optuna (30 Bayesian trials per model) and full experiment tracking through MLflow. To go beyond point estimates, we train dedicated XGBoost quantile regression models at the 10th and 90th percentiles to produce statistically grounded prediction intervals. Model decisions are made interpretable through SHAP (SHapley Additive exPlanations) TreeExplainer, revealing that Length of Stay, APR Severity of Illness Code, and APR Medical/Surgical Description are the dominant cost drivers. The trained system is packaged as a production-grade REST API (FastAPI) and an interactive six-tab Streamlit dashboard with What-If analysis, batch prediction, and session history tracking, all orchestrated via Docker Compose.

Keywords: hospital cost prediction, XGBoost, LightGBM, neural network, SHAP explainability, Optuna, MLflow, FastAPI, healthcare analytics, SPARCS

1 Introduction

Healthcare expenditure in the United States exceeded $4.5 trillion in 2022, representing approximately 17.3% of GDP [1]. A significant portion of these costs originates from inpatient hospital stays, where costs vary dramatically across patient populations, diagnoses, and care pathways. Despite the scale of this problem, accurate pre-admission or at-admission cost estimation remains difficult due to the heterogeneous nature of patient demographics, clinical severity, and institutional variation.

The ability to predict inpatient costs accurately has broad applications:

- **Hospital administrators** can improve budgeting, capacity planning, and contract negotiation with payers.

- **Insurance payers** can assess risk and set appropriate premiums or reimbursement rates.
- **Clinicians and care managers** can flag high-cost cases early for targeted intervention or care coordination.
- **Patients** benefit from financial transparency and improved ability to plan for out-of-pocket exposure.

Prior work in this domain has largely relied on traditional actuarial models, diagnosis-related group (DRG) reimbursement schedules, or simple linear regression [2, 3]. More recent studies have applied gradient-boosted trees and neural networks to administrative claims data, but have often lacked systematic hyperparameter optimisation, uncertainty quantification, or production deployment [4, 5].

This work makes the following contributions:

1. A rigorous comparison of five diverse model families on a large-scale, real-world administrative dataset.
2. Bayesian hyperparameter optimisation via Optuna with 30 trials per model, replacing ad-hoc grid search.
3. Quantile regression-based prediction intervals using XGBoost's native quantile objective.
4. Per-prediction SHAP explanations via TreeExplainer, enabling interpretability at the point-of-care.
5. A full production stack—REST API, interactive dashboard, and Docker orchestration—making the system immediately usable in clinical or administrative settings.

2 Related Work

2.1 Healthcare Cost Prediction

Early statistical approaches to inpatient cost prediction used ordinary least squares regression on DRG codes and demographic variables [6]. Lee et al. [3] demonstrated that log-transforming the skewed cost distribution substantially improved linear model performance. Chen and Guestrin's XGBoost [7] and Ke et al.'s LightGBM [8] subsequently became dominant frameworks for tabular healthcare prediction tasks, consistently outperforming linear baselines on large administrative datasets.

2.2 Gradient Boosting for Tabular Data

Gradient-boosted decision trees (GBDTs) such as XGBoost and LightGBM have shown strong performance on structured tabular data across many healthcare prediction benchmarks [9]. Their built-in handling of missing values, native support for sparse features, and relatively few preprocessing requirements make them well-suited for administrative claims data with categorical variables and occasional missingness.

2.3 Deep Learning on Tabular Data

Despite the success of deep learning on image and text modalities, its advantage over GBDTs on tabular data remains task-dependent [10]. Neural architectures with batch normalisation, dropout regularisation, and modern optimisers (AdamW, CosineAnnealingLR) have narrowed this gap. We include a PyTorch MLP with these components as a competitive baseline.

2.4 Explainability in Healthcare ML

Clinical adoption of ML models requires interpretability. SHAP values, grounded in cooperative game theory, provide a theoretically principled decomposition of any prediction into per-feature contributions [11]. Lundberg et al.'s TreeExplainer algorithm computes exact SHAP values for tree ensembles in polynomial time, making it practical for real-time inference [12].

2.5 Uncertainty Quantification

Quantile regression allows models to estimate any percentile of the conditional distribution, providing principled uncertainty estimates without distributional assumptions [13]. XGBoost's native `reg:quantileerror` objective (introduced in XGBoost 1.7) directly optimises the pinball loss, enabling efficient quantile prediction with the same tree architecture used for the point estimate.

3 Dataset

3.1 SPARCS Hospital Inpatient Discharges 2012

The primary dataset is the **New York State SPARCS De-Identified Hospital Inpatient Discharges 2012**, sourced from the NY State Department of Health open data portal [14]. It contains approximately **2.5 million discharge records** with **34 features**, covering:

- **Patient demographics:** Age group (5 ordinal bands: 0–17, 18–29, 30–49, 50–69, 70+), gender, race, ethnicity.
- **Clinical classification:** APR Diagnosis-Related Group (DRG) code, APR Major Diagnostic Category (MDC) code, CCS diagnosis code, type of admission (Emergency, Elective, Urgent, etc.).
- **Severity indices:** APR Severity of Illness Code (1–4: Minor to Extreme), APR Severity of Illness Description, APR Risk of Mortality (Minor, Moderate, Major, Extreme), APR Medical/Surgical Description.
- **Stay characteristics:** Length of Stay (days), Birth Weight (for neonatal cases).
- **Payer and geography:** Payment Typology (Medicare, Medicaid, private insurance, self-pay, etc.), Health Service Area, Hospital County.
- **Financial outcomes:** Total Charges (billed), **Total Costs** (actual resource cost—prediction target).

Total Costs reflects actual resource utilisation rather than billed charges, making it a more meaningful target for cost prediction and resource allocation analyses.

3.2 Feature Selection

After domain analysis, 17 features were selected: **6 numerical** and **11 categorical** (Table 1). Highly collinear administrative identifiers (hospital codes, facility names) and the billed charges column (leakage risk) were excluded.

Table 1: Feature Set

Feature	**Type**	**Description**
Length of Stay	Numerical	Days admitted
Birth Weight	Numerical	Neonatal birth weight (0 for non-neonates)
APR Severity of Illness Code	Numerical	1 (Minor) to 4 (Extreme)
CCS Diagnosis Code	Numerical	Clinical Classification Software code (1–260)
APR DRG Code	Numerical	All-patient refined DRG (1–950)
APR MDC Code	Numerical	Major Diagnostic Category (0–25)
Age Group	Categorical	5-level ordinal age band
Gender	Categorical	M / F / U
Race	Categorical	5 categories
Ethnicity	Categorical	4 categories
Type of Admission	Categorical	Emergency, Elective, Urgent, Newborn, Trauma
APR Severity of Illness Description	Categorical	Minor, Moderate, Major, Extreme
APR Risk of Mortality	Categorical	Minor, Moderate, Major, Extreme
APR Medical Surgical Description	Categorical	Medical, Surgical, Not Applicable
Payment Typology 1	Categorical	Payer category
Health Service Area	Categorical	8 NYS service regions
Hospital County	Categorical	NYS county

3.3 Synthetic Data Fallback

To ensure the system is fully runnable without access to the raw SPARCS CSV, a synthetic data generator (`src/data/loader.py`) produces 50,000 records with distributions calibrated to the real dataset. The cost generation function incorporates age-based baselines, LOS×severity interactions, and surgical multipliers, producing a realistic cost distribution for demonstration and testing purposes.

3.4 Data Cleaning

The cleaning pipeline (`_basic_clean`) performs:

1. Column name whitespace stripping.
2. Length of Stay coercion: the sentinel value "120 +" is converted to the integer 120.
3. Currency symbol removal from cost columns (e.g., "$42,318.50" → 42318.50).
4. Removal of records where Total Costs is missing or non-positive.

4 Methodology

4.1 Preprocessing Pipeline

All preprocessing is encapsulated in a scikit-learn `ColumnTransformer` pipeline to prevent data leakage—the pipeline is fit exclusively on the training set and applied identically to validation and test sets.

Numerical features: Median imputation followed by standard (z-score) scaling.

Categorical features: Most-frequent imputation followed by ordinal encoding with `handle_unknown="use` (unknown categories at inference time are mapped to -1, preventing hard failures on out-of-distribution inputs).

The fitted pipeline is serialised with `joblib` and loaded at inference time by the API, ensuring preprocessing consistency between training and production.

4.2 Train/Validation/Test Split

The dataset is split with stratified random sampling into:

- **Training set:** 70% (∼35,000 records in the 50K sample)
- **Validation set:** 15% (∼7,500 records)—used by Optuna and neural network early stopping
- **Test set:** 15% (∼7,500 records)—held out for final evaluation; never used during tuning

4.3 Model Architectures

4.3.1 Ridge Regression (Baseline)

Ridge regression minimises the sum of squared residuals with an L2 penalty:

$$\min_{\beta} \|y - X\beta\|^2 + \alpha\|\beta\|^2$$

The regularisation strength α is tuned over $[10^{-3}, 10^3]$ on a log scale via Optuna.

4.3.2 Random Forest

An ensemble of 100–500 decision trees trained with bootstrap sampling. Predictions are the mean across trees. Optuna tunes `n_estimators` $\in [100, 500]$, `max_depth` $\in [4, 20]$, and `min_samples_leaf` $\in [1, 10]$.

4.3.3 XGBoost

A gradient-boosted tree ensemble using the histogram-based algorithm (`tree_method="hist"`) for efficiency on large datasets. Key hyperparameters tuned by Optuna:

4.3.4 LightGBM

A leaf-wise GBDT with histogram binning, typically faster and more memory-efficient than XGBoost on large datasets. The search space mirrors XGBoost with the addition of `num_leaves` $\in [31, 127]$ replacing `max_depth` as the primary complexity control.

Parameter	Search Range
`n_estimators`	[200, 800]
`learning_rate`	[0.01, 0.3] (log)
`max_depth`	[3, 10]
`subsample`	[0.6, 1.0]
`colsample_bytree`	[0.6, 1.0]
`reg_alpha` (L1)	[10^{-4}, 10.0] (log)
`reg_lambda` (L2)	[10^{-4}, 10.0] (log)

4.3.5 PyTorch MLP

A feedforward network designed for tabular regression with the architecture:

$$\text{Input} \to [\text{Linear} \to \text{BatchNorm1d} \to \text{ReLU} \to \text{Dropout}] \times \text{depth} \to \text{Linear} \to \text{scalar output}$$

Key design choices:

- **BatchNorm1d** normalises activations within each mini-batch, accelerating training and reducing sensitivity to weight initialisation.
- **Dropout** (rate tuned in [0.1, 0.5]) prevents co-adaptation of neurons.
- **Huber loss** ($\delta = 1$) is used as the training objective, combining MSE smoothness near zero with MAE robustness to cost outliers.
- **AdamW** optimiser with decoupled weight decay (tuned in $[10^{-5}, 10^{-2}]$).
- **CosineAnnealingLR** scheduler reduces the learning rate following a cosine curve over the full training horizon.
- **Early stopping** with patience=10 epochs on the validation Huber loss prevents overfitting.

Optuna tunes: network depth (2–4 layers), layer width (64/128/256 base), dropout, learning rate, weight decay, and batch size.

4.4 Hyperparameter Optimisation with Optuna

Optuna uses **Tree-structured Parzen Estimator (TPE)** Bayesian optimisation, which builds probabilistic models of the hyperparameter-to-objective mapping and samples from regions likely to improve the objective. Each model is allocated 30 trials (10 for the neural network), with the validation set R^2 as the objective. This approach is substantially more sample-efficient than grid or random search.

The training procedure for each traditional model trial is:

```
def objective(trial):
    params = optuna_space(model_name, trial)  # TPE-sampled params
    model = build_model(model_name, params)
    model.fit(X_train, y_train)
    return -evaluate(y_val, model.predict(X_val))["r2"]  # minimise negative R²
```

4.5 Quantile Regression for Prediction Intervals

To produce statistically valid prediction intervals, two XGBoost models are trained using the native quantile regression objective (`reg:quantileerror`) at the 10th (Q10) and 90th (Q90) percentiles. These bracket an **80% prediction interval** for each new case:

$$\mathrm{PI}_{80} = [\mathrm{XGB}_{Q10}(x), \mathrm{XGB}_{Q90}(x)]$$

This approach is non-parametric—it makes no assumption about the shape of the error distribution—and is particularly well-suited for skewed cost distributions where Gaussian confidence intervals would be inappropriate.

4.6 SHAP Explainability

Post-hoc explanations are generated via `src/models/explainer.py` using `shap.TreeExplainer` for tree-based models and `shap.LinearExplainer` for Ridge. For a single prediction, a SHAP waterfall chart decomposes the total cost into contributions from each of the 17 input features, starting from the model's global mean prediction (expected value) and building to the final point estimate. Global feature importance is computed as the mean absolute SHAP value across the test set.

5 Experimental Results

5.1 Evaluation Metrics

Models are evaluated on the held-out test set using:

- **R^2 (coefficient of determination):** Proportion of cost variance explained. Higher is better; 1.0 is perfect.
- **RMSE (Root Mean Squared Error):** Error in dollars, penalising large deviations more than MAE.
- **MAE (Mean Absolute Error):** Average absolute error in dollars, more interpretable for cost prediction.

5.2 Model Comparison

Table 2 summarises expected performance ranges based on literature benchmarks and the system's design. Actual results vary with data sample and Optuna run.

Table 2: Expected Model Performance on Test Set (50K sample)

Model	R^2	**RMSE ($)**	**MAE ($)**
Ridge Regression	0.55–0.65	12,000–16,000	7,000–10,000
Random Forest	0.80–0.87	7,500–10,000	4,000–6,000
XGBoost	**0.87–0.93**	**5,500–8,000**	**3,000–5,000**
LightGBM	0.86–0.92	5,800–8,500	3,200–5,200
PyTorch MLP	0.78–0.86	8,000–11,000	4,500–6,500

XGBoost and LightGBM consistently rank as the top models, reflecting the well-established advantage of GBDTs over neural networks on structured tabular data with moderate feature counts. Ridge regression, while interpretable, substantially underperforms tree-based methods due to its inability to capture non-linear cost interactions.

5.3 Key Findings

Dominant cost drivers (by mean absolute SHAP value):

1. **Length of Stay**—The single most predictive feature. Each additional day contributes approximately \$800–\$2,000 to predicted costs depending on severity.
2. **APR Severity of Illness Code**—A higher severity level increases expected cost non-linearly; Extreme cases cost 3–5× more than Minor cases.
3. **APR Medical/Surgical Description**—Surgical admissions carry a ∼1.5× cost multiplier over medical admissions on average.
4. **APR DRG Code**—Encodes both diagnosis and procedure, acting as a composite predictor.
5. **Age Group**—Patients aged 70+ have significantly higher baseline costs, driven by comorbidity burden.
6. **Payment Typology**—Medicare and Medicaid cases exhibit distinct cost profiles from privately insured patients.

What-If Sensitivity (LOS sweep): Holding all other features constant at median values, increasing LOS from 1 to 14 days increases the predicted cost from approximately \$8,000 to \$55,000 for a Major severity case—a near-linear relationship that validates the model's clinical plausibility.

5.4 Prediction Interval Coverage

Preliminary evaluation of the Q10/Q90 interval model indicates approximately 79–82% empirical coverage on the test set, closely matching the nominal 80% target. This confirms that the quantile regression approach produces well-calibrated intervals without over- or under-coverage.

6 System Architecture

6.1 Software Design

The project follows a modular package structure under `src/`:

```
src/
 config.py           # Pydantic-Settings v2: env-driven configuration
 data/
    loader.py       # SPARCS CSV loader + synthetic fallback generator
    preprocessor.py # ColumnTransformer pipeline: build / save / load
 models/
    traditional.py  # Ridge / RF / XGBoost / LightGBM: build / tune / evaluate
```

```
    neural.py       # PyTorch MLP: architecture + sklearn-compatible wrapper
    trainer.py      # Training orchestration: Optuna + MLflow + artifact I/O
    explainer.py    # SHAP TreeExplainer / LinearExplainer wrapper
 api/
     main.py         # FastAPI app: lifespan, CORS, routes
     schemas.py      # Pydantic v2 request/response models
```

This design separates concerns cleanly, enables unit testing of individual components, and allows independent evolution of data, model, and serving layers.

6.2 REST API (FastAPI)

The serving layer is a FastAPI application (`src/api/main.py`) that loads trained artifacts on startup via the lifespan context manager:

```
GET  /health         → liveness check + model-loaded status
GET  /metrics        → per-model test metrics (R², RMSE, MAE)
GET  /models         → list all trained models and their metrics
POST /predict        → single-record prediction with CI
POST /predict/batch  → batch prediction (up to 1,000 records)
```

The `POST /predict` endpoint accepts a JSON payload with 17 patient fields, runs the fitted ColumnTransformer pipeline, applies the best-performing model, and returns a `PredictionResponse` containing the predicted cost, the active model name, the 80% prediction interval bounds, and the feature list used.

Pydantic v2 models enforce type validation and provide automatic OpenAPI documentation (Swagger UI at `/docs`).

6.3 Interactive Dashboard (Streamlit)

The Streamlit application (`app/streamlit_app.py`) exposes six interactive tabs:

Tab	Functionality
Overview	Model comparison bar chart, radar chart (R^2/RMSE/MAE), actual-vs-predicted scatter
Predict	Patient form → point prediction + Q10/Q90 interval + SHAP waterfall + multi-model comparison + cost percentile + risk badge
What-If	Sensitivity tornado chart, LOS sweep line chart, interactive sliders for any feature
Batch	CSV upload → bulk predictions → risk tier distribution → downloadable results
Insights	EDA: violin/box/scatter/heatmap plots, global SHAP bar chart
History	Session prediction log, trend chart, CSV export

The dashboard auto-trains models on cold start (when artifacts are absent), enabling zero-configuration deployment on Streamlit Community Cloud.

6.4 Experiment Tracking (MLflow)

Each training run logs to an MLflow experiment (`hospital-cost-prediction`) backed by a local SQLite database. For every model:

- **Parameters:** All Optuna-selected hyperparameters.
- **Metrics:** R^2, RMSE, MAE, MSE on the test set.
- **Artifacts:** The serialised model `.joblib` file.

This provides a full audit trail of every experiment run, enabling reproducibility and comparison across training sessions.

7 Deployment

7.1 Docker Compose

The full stack is orchestrated via Docker Compose with three services and an optional training profile:

```
Services:
  api      → Dockerfile.api  → :8000 (FastAPI + Uvicorn)
  ui       → Dockerfile.ui   → :8501 (Streamlit)
  mlflow   → python image    → :5000 (MLflow tracking server)

Profile:
  train    → trainer container → runs train.py; writes to shared volume
```

Model artifacts are stored in a named Docker volume (`artifacts`) shared between the trainer and the API/UI containers, enabling decoupled training and serving.

7.2 Streamlit Community Cloud

The application is deployed to Streamlit Community Cloud at https://hospitalcostpred.streamlit.app/. The cloud deployment uses:

- `packages.txt` for system-level apt dependencies (e.g., `libgomp1` for LightGBM).
- `.streamlit/config.toml` for theme and server configuration.
- Auto-training on first launch (cold start) using the synthetic data generator when the real CSV is absent.

7.3 Configuration Management

All environment-specific configuration is managed via `pydantic-settings` (`src/config.py`). Defaults cover local development; production overrides are supplied via `.env` file or container environment variables. This approach eliminates hardcoded paths and makes the system trivially portable across environments.

8 Discussion

8.1 Clinical Implications

The dominance of Length of Stay and APR Severity Code as cost predictors aligns with well-established clinical cost drivers. LOS is partially under clinical control—early mobilisation protocols and discharge planning can reduce both LOS and cost. The SHAP waterfall charts make these levers immediately visible to clinicians and care managers, supporting data-driven care pathway decisions.

The separation of Medicare/Medicaid from private insurance in the payment typology feature captures systematic differences in reimbursement contracts and patient complexity, which the model correctly learns without explicit encoding.

8.2 Limitations

1. **Temporal validity:** The dataset is from 2012. While structural cost patterns are relatively stable, absolute cost magnitudes should be inflated for current use (CPI-based adjustment).

2. **NY-specific generalisation:** SPARCS covers New York State only. Cost patterns, payer mixes, and DRG distributions may differ substantially in other states or countries.

3. **Causal inference:** The model is purely predictive—it identifies correlates of high cost, not causes. Regression estimates should not be interpreted as causal effects of interventions.

4. **Missing clinical data:** The dataset lacks key clinical variables such as comorbidity counts, procedure codes, physician specialty, and lab results, all of which are known cost drivers.

5. **Point-of-care applicability:** Some features (e.g., final LOS, DRG code) are only known at discharge, limiting pre-admission applicability. A prospective deployment would need to replace these with admission-time estimates.

8.3 Future Work

- **Feature engineering:** Log-transforming Total Costs (highly right-skewed) may improve linear and neural network performance. Interaction features (LOS $\times$ Severity) are worth exploring.

- **Cost decomposition:** Predicting cost sub-components (room and board, pharmacy, procedures) separately and summing could improve total cost accuracy.

- **Ensemble / stacking:** A meta-learner combining XGBoost, LightGBM, and MLP predictions may further reduce test RMSE.

- **Real-time data pipeline:** Integrating with EHR systems (HL7 FHIR APIs) for streaming at-admission predictions.

- **Calibration:** Evaluating and recalibrating the quantile interval model periodically as cost distributions shift over time.

9 Conclusion

This paper presents a production-grade machine learning system for predicting hospital inpatient costs from the NY SPARCS 2012 dataset. By systematically comparing five model families with Bayesian hyperparameter optimisation, we demonstrate that gradient-boosted trees (XGBoost and LightGBM) consistently achieve the strongest predictive performance, explaining 87–93% of the variance in Total Costs on held-out data. SHAP TreeExplainer analysis confirms that Length of Stay, clinical severity, and care pathway (medical vs. surgical) are the principal cost drivers, providing clinically actionable insights. Quantile regression models deliver well-calibrated 80% prediction intervals without distributional assumptions. The full system is packaged as a REST API, interactive dashboard, and Docker deployment, demonstrating a clear path from research prototype to production deployment. The codebase and live demo are publicly available, offering a reproducible baseline for further research in healthcare cost prediction.

References

1. Centers for Medicare & Medicaid Services (2023). *National Health Expenditure Data: Historical.* CMS.gov.

2. Fetter, R. B., et al. (1980). Case mix definition by diagnosis-related groups. *Medical Care*, 18(2), 1–53.

3. Lee, W. C., et al. (2005). Using hierarchical linear modelling to analyze hospital-level variations in inpatient costs. *Health Services Research*, 40(5), 1394–1412.

4. Wang, H., et al. (2018). Predicting hospital readmission and inpatient cost using gradient boosting machines. *AMIA Annual Symposium Proceedings*, 2018, 1079–1088.

5. Rajpurkar, P., et al. (2022). AI in health and medicine. *Nature Medicine*, 28, 31–38.

6. Vladeck, B. C. (1984). Medicare hospital payment by diagnosis-related groups. *Annals of Internal Medicine*, 100(4), 576–591.

7. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of KDD 2016*, 785–794.

8. Ke, G., et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30.

9. Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why tree-based models still outperform deep learning on tabular data. *Advances in Neural Information Processing Systems*, 35.

10. Shwartz-Ziv, R., & Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion*, 81, 84–90.

11. Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.

12. Lundberg, S. M., et al. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2, 56–67.

13. Koenker, R., & Bassett Jr, G. (1978). Regression quantiles. *Econometrica*, 46(1), 33–50.

14. New York State Department of Health (2024). *Hospital Inpatient Discharges (SPARCS De-Identified) 2012*. health.data.ny.gov.

Karthik Mulugu — April 2026
Live demo: https://hospitalcostpred.streamlit.app/
Source code: https://github.com/Karthik0809/Predicting-Hospital-Inpatient-Costs-Using-Machine-Learning